Listing 11.12 The `filter` clause

```
GET products/_search
{
  "query": {
    "bool": {
      "filter": [{
          "term": {
            "product.keyword": "TV"
          },
          {
            "range": {
              "price": {
                "gte": 500,
                "lte": 1000
              }
            }
          }
        }
      ]
    }
  }
}
```

Executes the query in a filter context implicitly

The following snippet shows the results of this query. Note that the result scores are zero:

```
 "hits" : [{
      ...
      "_score" : 0.0,
      "_source" : {
        "product" : "TV",
        "colour" : "silver",
        "brand" : "Samsung"
      }
    },
    {
      ...
      "_score" : 0.0,
      "_source" : {
        "product" : "TV",
        "colour" : "black",
        "brand" : "Samsung"
      }
    }
    ...
]
```

The `filter` clause provides no scoring. Because scoring is not required for the output, Elasticsearch can cache the query/results, which benefits the application's performance.

We usually combine a `filter` clause with a `must` clause. The results of the `must` clause are fed through the `filter` clause, which filters non-fitting data. The next example shows this approach.